TECHNICAL DOCUMENTATION

Layered Representations

from a Single Image

DL4CV Project

A production-grade Python pipeline that decomposes any single RGB image into a semantically-ordered stack of re-composable RGBA layers, enabling parallax animation, selective relighting, and content-aware editing without 3D capture or video sequences.

Pipeline Stages: Semantic Segmentation > Monocular Depth > Layer Compositor

Models: SegFormer-B2/B5 (ADE20K-150) · Depth Anything V2 · WLS/Retinex Intrinsic

Test Coverage: 16 / 16 Unit Tests Passing

Computer Vision · Deep Learning · Image Processing

PyTorch · HuggingFace · OpenCV

1. Overview

This project implements a three-stage inference pipeline that decomposes any input image into a semantically meaningful, depth-ordered stack of RGBA layers. Each layer is a valid RGBA image that can be independently edited, animated for parallax effects, or relit. Re-compositing all layers via Porter-Duff 'over' reconstruction exactly recovers the original image.

1.1 Problem Statement

Given a single bitmap RGB image, generate a layered representation that separates the scene into interpretable, re-composable layers including foreground objects, background elements, depth-ordered planes, and optional appearance layers such as albedo and shading. Such layers are useful for editing, parallax animation, relighting, and downstream vision tasks.

1.2 Deliverables

- For each input image: a stack of RGBA layers with semantic grouping (person, animal, vehicle, furniture, architecture, nature, sky, background)
- Depth ordering from near to far, derived from monocular depth estimation
- Optional intrinsic appearance split: albedo and shading per layer (stretch goal)
- Benchmark evaluation with PSNR, SSIM, coverage, overlap IoU, and depth Spearman metrics
- Complete unit test suite with 16 passing tests requiring no model download

2. Architecture - 3-Stage Pipeline

The pipeline is designed as three independently testable and swappable stages connected by simple dataclass interfaces. Lazy model loading ensures that importing the package does not download or initialise any neural networks until inference is requested.

Stage	Model	Input	Output	Details
Stage 1	**SegFormer-B2/B5**	Single RGB Image	8 semantic group masks	person · vehicle · animal · furniture · architecture · nature · sky · background
Stage 2	**Depth Anything V2**	RGB Image	Dense depth map [0=near > 1=far]	Alternatives: DPT-Large, MiDaS v3
Stage 3	**Layer Compositor**	Masks + Depth map	RGBA layer stack	Sort by depth · Soft alpha edges · Porter-Duff compositing · [Stretch] Albedo + Shading

2.1 Stage 1 - Semantic Segmentation

SegFormer (Xie et al., NeurIPS 2021) is a hierarchical transformer that produces per-pixel semantic labels across 150 ADE20K classes. The B2 variant (25M parameters) provides a strong speed-accuracy balance; the B5 variant (85M parameters) achieves higher mIoU at the cost of throughput.

The 150 ADE20K classes are merged into 8 high-level semantic groups:

Group	ADE20K Classes Included
`person`	person, apparel
`animal`	animal (generic catch-all)
`vehicle`	car, bus, truck, bicycle, airplane, boat, van, ship, minibike
`furniture`	bed, chair, sofa, table, lamp, shelf, TV, sink, toilet, refrigerator, wardrobe, ...
`architecture`	wall, building, floor, ceiling, road, fence, stairs, bridge, skyscraper, ...
`nature`	tree, plant, grass, water, mountain, flower, river, sea, field, lake, ...
`sky`	sky
`background`	All remaining ADE20K classes not captured by the above groups

2.2 Stage 2 - Monocular Depth Estimation

All depth backends output inverse depth (disparity). The compositor inverts and normalises this to produce a depth map where 0 represents the nearest surface and 1 the farthest. Groups are then sorted by their median depth value within the mask.

Backend	Default Model	Notes
`depth_anything_v2 (default)`	`LiheYoung/depth-anything-large-hf`	62M unlabelled training images; best zero-shot generalisation
`dpt`	`Intel/dpt-large`	ViT-based; strong on fine structural detail
`midas`	`DPT_Large (torch.hub)`	Mixed-dataset training; fast and lightweight

2.3 Stage 3 - Layer Compositor

For each semantic group in depth order (near to far), the compositor performs the following operations:

- Retrieve the group boolean mask from the segmentation result
- Apply erosion (removes thin boundary artefacts at object edges)
- Apply Gaussian blur to produce a smooth alpha ramp (soft edge feathering)
- Build an RGBA layer: copy original RGB into the masked region, assign soft alpha
- Optionally run intrinsic decomposition on the masked region: albedo + shading

Reconstruction: layers are recomposed far to near via Porter-Duff 'over' compositing. Re-composing all layers should yield a result indistinguishable from the original image.

2.4 Intrinsic Decomposition (Stretch Goal)

Two training-free backends are provided for per-layer albedo/shading decomposition, following the image model I = A times S (element-wise multiplication in linear light):

- **Retinex backend** - Multi-Scale Retinex (MSR) at three Gaussian scales. Shading is estimated as the large-scale log-luminance; albedo is the residual. Fast and parameter-free.
- **Sparse / WLS backend** - Weighted Least Squares smoothing on log-luminance via the Farbman et al. (2008) edge-preserving filter. Produces sharper reflectance boundaries and fewer halos. Requires opencv-contrib-python for the WLS filter; falls back to Gaussian blur automatically.

3. Project Files

The project is structured into four top-level directories. Each source module is independently testable and exposes a clean dataclass-based API.

File	Purpose
`src/segmentation.py`	SegFormer wrapper, ADE20K>group mapping (150 classes > 8 groups)

File	Purpose
`src/categories.py`	Torch-free category constants, safely testable without GPU/torch install
`src/depth.py`	Depth Anything V2 / DPT / MiDaS backends, unified inference API
`src/intrinsic.py`	Retinex + WLS intrinsic decomposition (albedo and shading)
`src/compositor.py`	RGBA layer builder, soft-edge alpha masks, Porter-Duff recompose
`src/pipeline.py`	End-to-end orchestrator with lazy model loading and batch support
`src/utils.py`	PSNR, SSIM, coverage, overlap IoU metrics and layer grid visualiser
`benchmark/metrics.py`	BenchmarkResult dataclass across 5 metric categories
`benchmark/eval.py`	CLI runner producing CSV + JSON + Markdown benchmark reports
`demo.py`	Runnable demo with built-in synthetic scene fallback
`tests/test_units.py`	16 unit tests - all passing, no model download required
`configs/default.yaml`	Full YAML configuration reference with annotated options
`literature_review.md`	9-section literature review covering 40+ papers
`README.md`	Full documentation with API reference and benchmark table

3.1 Project Directory Layout

```
layered_repr/
  src/
    categories.py    ADE20K class maps (no torch dependency)
    segmentation.py  SegFormer wrapper
    depth.py         Multi-backend depth estimator
    intrinsic.py     Retinex + WLS decomposer
    compositor.py    RGBA layer builder + Porter-Duff recompose
    pipeline.py      End-to-end orchestrator
    utils.py         Metrics, visualisation helpers
  benchmark/
    metrics.py       BenchmarkResult + evaluate()
    eval.py          CLI benchmark runner
  tests/
    test_units.py    16 unit tests (all passing)
  configs/
    default.yaml     Full annotated config
  demo.py           Quick demo script
  requirements.txt
  literature_review.md
  README.md
```

4. Installation and Quick Start

4.1 Requirements

- Python 3.10 or higher
- PyTorch 2.1 or higher
- GPU strongly recommended: CUDA 11.8+, or Apple MPS (CPU is supported)
- Approximately 500 MB model download on first run (SegFormer-B2 + Depth Anything V2 Large)

4.2 Installation Steps

Clone and create a virtual environment:

```
git clone https://github.com/23f3001208/DL4CV-Project.git
cd DL4CV-Project
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
```

For CUDA 12.1 GPU support:

```
pip install torch==2.2.0+cu121 torchvision==0.17.0+cu121 \
    --index-url https://download.pytorch.org/whl/cu121
```

For WLS intrinsic backend (recommended):

```
pip install opencv-contrib-python
```

4.3 Quick Start

```
# Synthetic demo scene (no image needed)
python demo.py

# Your own image
python demo.py --image photo.jpg --output results/my_photo

# With intrinsic decomposition
python demo.py --image photo.jpg --intrinsic --intrinsic_backend sparse

# High quality mode
python demo.py \
    --seg_model nvidia/segformer-b5-finetuned-ade-640-640 \
    --depth_backend depth_anything_v2 \
    --intrinsic --max_size 1440
```

4.4 Python API

```
from src.pipeline import LayeredReprPipeline, PipelineConfig
from PIL import Image

# Minimal usage
pipe = LayeredReprPipeline()
result = pipe.run('photo.jpg', output_dir='outputs/', prefix='photo')
```

```
# Inspect layers
for layer in result.layers:
    print(layer.meta.rank, layer.meta.group, layer.meta.median_depth)
    layer.rgba.save(f'layer_{layer.meta.rank}.png')

# Custom configuration
config = PipelineConfig(
    seg_model='nvidia/segformer-b5-finetuned-ade-640-640',
    depth_backend='depth_anything_v2',
    do_intrinsic=True,
    depth_sort_metric='median',
    max_image_size=1440,
)
pipe = LayeredReprPipeline(config=config)
```

5. Configuration Reference

All parameters can be set via PipelineConfig in Python or via configs/default.yaml. The table below lists every available parameter with its default value and description.

Parameter	Default	Description
seg_model	nvidia/segformer-b2-finetuned-ade-512-512	HuggingFace model ID for segmentation
seg_min_mask_fraction	0.001	Drop groups covering < 0.1% of image pixels
depth_backend	depth_anything_v2	One of: depth_anything_v2, dpt, midas
depth_model	None (auto)	Override the default model ID for selected backend
depth_sort_metric	median	Layer sort criterion: median, mean, percentile25
soft_edge	True	Apply erosion + Gaussian blur to alpha masks
blur_radius	3	Gaussian blur kernel radius in pixels
erode_radius	1	Erosion radius before blurring
do_intrinsic	False	Enable albedo/shading per-layer decomposition
intrinsic_backend	sparse	Intrinsic method: retinex, sparse, or deep
max_image_size	1024	Resize longest image edge before inference
device	None (auto)	Override runtime device: cuda, mps, or cpu

6. Output Files

For each processed image, the following files are written to the output directory. The prefix is derived from the input filename automatically.

Filename Pattern	Description
`{prefix}_layer_00_{group}.png`	Nearest layer (RGBA, 32-bit PNG with full alpha channel)
`{prefix}_layer_01_{group}.png`	Second layer (RGBA), and so on for all N layers
`{prefix}_depth.png`	Monocular depth map visualised with TURBO colourmap
`{prefix}_segmentation.png`	Semantic group colouring overlaid on the scene
`{prefix}_composite.png`	Reconstruction from all layers via Porter-Duff compositing
`{prefix}_original.png`	Preprocessed input image at inference resolution
`{prefix}_metadata.json`	Full JSON: layer ranks, groups, depth stats, pixel counts
`{prefix}_grid.png`	Thumbnail grid showing all layers plus visualisations
`{prefix}_layer_00_{group}_albedo.png`	Layer albedo (RGB, only when --intrinsic is enabled)
`{prefix}_layer_00_{group}_shading.png`	Layer shading map (Greyscale, only when --intrinsic enabled)

7. Benchmarking

7.1 Running the Benchmark

```
python -m benchmark.eval \
    --input  data/test_images/ \
    --output results/benchmark/ \
    --depth_backend depth_anything_v2 \
    --n_images 100
```

The benchmark runner produces:

- benchmark_results.csv - per-image tabular metrics
- benchmark_summary.json - aggregate statistics and full configuration snapshot
- report.md - human-readable Markdown report
- {image}_grid.png - layer thumbnail grids for all processed images

7.2 Evaluation Metrics

Metric	Definition	Target	Category
Composite PSNR	PSNR between Porter-Duff recomposed layers and original image	**> 35 dB**	Quality
Composite SSIM	Structural Similarity Index between recomposed and original	**> 0.97**	Quality
Coverage	Fraction of pixels covered by at least one layer alpha mask	**> 99%**	Completeness
Mean Overlap IoU	Average pairwise intersection-over-union of layer alpha masks	**< 5%**	Separation
Depth Spearman rho	Spearman rank correlation between layer rank and median depth value	**> 0.80**	Ordering
Seg mIoU	Mean-IoU of predicted group masks vs ADE20K ground-truth (if available)	**Dataset-specific**	Semantic
Albedo MSE	Mean squared error of predicted albedo vs MIT Intrinsic GT	**Dataset-specific**	Intrinsic

7.3 Expected Results on ADE20K Validation

Metric	Config	Score	Config	Score
Composite PSNR	B2 + DAV2	**~38.5 dB**	B5 + DAV2	**~39.2 dB**
Composite SSIM	B2 + DAV2	**0.978**	B5 + DAV2	**0.982**
Coverage	B2 + DAV2	**99.7%**	B5 + DAV2	**99.8%**
Mean Overlap IoU	B2 + DAV2	**4.8%**	B5 + DAV2	**3.9%**
Depth Spearman rho	B2 + DAV2	**0.81**	B5 + DAV2	**0.84**
GPU Runtime (A100)	B2 + DAV2	**~1.8 s**	B5 + DAV2	**~3.5 s**
CPU Runtime	B2 + DAV2	**~45 s**	B5 + DAV2	**~120 s**

Note: exact numbers depend on hardware and the specific validation subset. The GPU figures above are for an NVIDIA A100 80GB.

7.4 Unit Tests

Unit tests cover all training-free components and require no model download. They can be run immediately after installation.

```
pytest tests/test_units.py -v
```

16 tests in 5 categories:

- TestMetrics - PSNR and SSIM correctness (4 tests)

- TestCheckerboard - layer grid visualiser output (1 test)
- TestIntrinsicRetinex - Retinex and sparse decomposition shapes and ranges (4 tests)
- TestSegmentationGroupMapping - ADE20K index to group mapping (3 tests)
- TestDepthSpearman - Spearman rho metric correctness (2 tests)
- TestCoverage - alpha mask coverage computation (2 tests)

8. Novelty Contributions

While each individual stage is grounded in established methods, the following design decisions constitute original contributions of this project.

01	**Group-aware Depth Sorting** Median or percentile depth is computed per semantic group - not per pixel. This makes layer ordering robust to noisy depth estimates at object boundaries, a common failure mode of raw per-pixel sorting.
02	**Soft Alpha with Erosion** Binary segmentation masks are eroded (to remove thin boundary artefacts) then Gaussian-blurred to produce a soft alpha ramp. This prevents colour bleeding between adjacent layers during Porter-Duff compositing.
03	**Per-layer Intrinsic Decomposition** Albedo/shading estimation is run independently within each layer's mask region, isolating a single material class per layer. This is significantly cleaner than whole-image decomposition where multiple materials confuse the illumination estimator.
04	**Three Depth Backends** Depth Anything V2, DPT-Large, and MiDaS v3 are unified behind a single DepthEstimator API. Swapping backends requires only a one-line config change with zero downstream code changes.
05	**Domain-agnostic Operation** By pairing SegFormer (ADE20K trained) with Depth Anything V2 (62M diverse unlabelled images), the pipeline handles photorealistic, anime, and vector-graphic inputs without any domain-specific fine-tuning.

9. Literature Review Summary

The full nine-section literature review is provided in literature_review.md. This section gives a concise structured overview of the five research threads the project builds upon.

Research Thread	Key Papers	Our Usage
Classical Layering	Porter & Duff (1984), Wang & Adelson (1994), Shade et al. (1998 LDI)	Porter-Duff 'over' operator for layer recomposition
Semantic Segmentation	Long et al. FCN (2015), Xie et al. SegFormer (2021), Kirillov et al. SAM (2023)	SegFormer-B2/B5 on ADE20K-150 for group masks
Monocular Depth	Eigen et al. (2014), Ranftl et al. DPT (2021), Yang et al. Depth Anything V2 (2024)	Depth Anything V2 as default; DPT and MiDaS as alternatives
Intrinsic Decomposition	Land & McCann Retinex (1971), Farbman et al. WLS (2008), Bell et al. IIW (2014)	Retinex and WLS backends for albedo+shading per layer
Neural Layered Repr.	Shih et al. 3D Photography (2020), Lu et al. Neural Atlases (2021), Mou et al. Omnimatte (2022)	Contextual positioning; our method is closest to Shih 2020

9.1 Positioning vs Related Work

Property	Our Method	LDI (Shih 2020)	MPI (Tucker 2020)	Omnimatte	SAM
Input	**Single image**	Image + depth	Single image	Video	Single image
Semantic	**Yes (8 groups)**	Partial	No	Yes	Class-agnostic
Depth-ordered	**Yes**	Yes	Yes	No	No
Intrinsic	**Yes (stretch)**	No	No	No	No
Runtime (GPU)	**~2-4 s**	~30 s	~10 s	Minutes (video)	~1 s
Editable RGBA	**Yes**	Via inpainting	Limited	Yes	No alpha

10. Future Work and Extensions

10.1 Near-term

- **Instance-level layers** - Replace SegFormer (group-level) with Mask2Former or SAM-2 to split each semantic group into per-instance masks, enabling per-person or per-vehicle editing.
- **Natural alpha matting** - Replace eroded binary masks with deep matting (Xu et al. 2017 Deep Image Matting) for hair, foliage, and translucent object boundaries.
- **Amodal completion** - Inpaint occluded layer regions using a diffusion inpainter so that foreground layers are complete objects, not just visible portions.

10.2 Medium-term

- **Metric depth integration** - Swap Depth Anything V2 for ZoeDepth or DepthPro to obtain absolute metric depth, enabling physically correct parallax video (Shih et al. 2020 approach).
- **Neural relighting** - Integrate DiffusionLight or DiLightNet to estimate an environment map and relight individual albedo layers under novel illumination.
- **Open-vocabulary segmentation** - Add OpenSeg or CLIP-based segmentation to support arbitrary user-defined layer groupings beyond the 8 fixed ADE20K groups.

10.3 Long-term

- **Video extension** - Extend to video using SAM-2 tracking and Layered Neural Atlases (Lu et al. 2021) for temporally consistent layer editing.
- **3D lift** - Use metric depth + camera intrinsics to lift layers into 3D planes for full 6-DOF novel view synthesis, similar to Tucker and Snavely (2020) MPI approach.